

EXPERIMENT - 35

1. Problem Statement

Write a menu-driven C program to read a variable-length square integer array and perform the following operations using separate functions:

- Determine whether all the elements on its main diagonal are distinct.
- Find the sum of its main diagonal elements.
- Display its elements using a diagonal traversal. Begin with the element in the top-left corner and traverse successive diagonals alternately in the upward-right and downward-left directions.
- Exit.

For the matrix 1 2 3 / 4 5 6 / 7 8 9 the main diagonal elements are 1 5 9 and all of them are distinct, their sum is 15, and the diagonal traversal is 1 2 4 7 5 3 6 8 9.

2. Algorithm

Main Program

Step 1: START

Step 2: Read n and declare the variable length array a of size $n \times n$, then read its elements.

Step 3: Display the menu and read choice.

Step 4: Using a switch statement on choice:

 If choice is 1, call `diagonalDistinct()`.

 If choice is 2, call `diagonalSum()`.

 If choice is 3, call `diagonalTraversal()`.

 If choice is 4, display "Exiting the program."

Step 5: Repeat from Step 3 while choice is not equal to 4.

Step 6: STOP

Function `diagonalDistinct()`

Step 1: Display $a[i][i]$ for $i = 0$ to $n - 1$.

Step 2: Set `distinct = 1`.

Step 3: Repeat for $i = 0$ to $n - 1$:

 Repeat for $j = i + 1$ to $n - 1$: if $a[i][i]$ is equal to $a[j][j]$, set `distinct = 0`.

Step 4: If `distinct` is 1, display that all the main diagonal elements are distinct, otherwise display that they are not.

Step 5: Return to the calling function.

Function `diagonalSum()`

Step 1: Set `sum = 0`.

Step 2: Repeat for $i = 0$ to $n - 1$: add $a[i][i]$ to `sum`.

Step 3: Display `sum`.

Step 4: Return to the calling function.

Function `diagonalTraversal()`

Step 1: Repeat for $d = 0$ to $2n - 2$:

Set low to $d - n + 1$ if that is greater than 0, otherwise to 0.

Set high to d if d is less than $n - 1$, otherwise to $n - 1$.

If d is even, repeat for $i = \text{high}$ down to low: display $a[i][d - i]$.

If d is odd, repeat for $i = \text{low}$ up to high: display $a[i][d - i]$.

Step 2: Return to the calling function.

3. Test Case Table

#	Operation	Input	Expected Output	Actual Output	Result
1	1. Distinct check	3×3 , 1 2 3 / 4 5 6 / 7 8 9	Main diagonal 1 5 9, all distinct	1 5 9, distinct	PASS
2	2. Diagonal sum	the same matrix	Sum of main diagonal elements: 15	15	PASS
3	3. Traversal	the same matrix	1 2 4 7 5 3 6 8 9	Matches the example	PASS
4	1. Distinct check	3×3 , 1 2 3 / 4 1 6 / 7 8 9	Main diagonal 1 1 9, not all distinct	1 1 9, not distinct	PASS
5	2. Diagonal sum	the same matrix	Sum is $1 + 1 + 9 = 11$	11	PASS
6	3. Traversal	4×4 holding 1 to 16	1 2 5 9 6 3 4 7 10 13 14 11 8 12 15 16	Same order	PASS
7	2. Diagonal sum	the same 4×4 matrix	Sum is $1 + 6 + 11 + 16 = 34$	34	PASS
8	4. Exit	choice = 4	Exiting the program, and the loop ends	Loop ended	PASS

4. Source Code

diagonal.c

```
/* Experiment 35. One square matrix is read at the start and all three
options work on that same matrix. The array is variable length,
because its order is only known at run time. */

#include <stdio.h>

void diagonalDistinct(int n, int a[n][n]);
void diagonalSum(int n, int a[n][n]);
void diagonalTraversal(int n, int a[n][n]);

int main(void)
{
    int n, i, j, choice;

    printf("Enter the order of the square matrix: ");
    scanf("%d", &n);

    int a[n][n];

    printf("Enter the matrix elements:\n");
    for (i = 0; i < n; i++)
        for (j = 0; j < n; j++)
            scanf("%d", &a[i][j]);

    do {
        printf("\nMENU\n");
        printf("1. Check whether all the main diagonal elements are distinct\n");
        printf("2. Find the sum of the main diagonal elements\n");
        printf("3. Display the elements using a diagonal traversal\n");
        printf("4. Exit\n");
        printf("Enter your choice: ");
        scanf("%d", &choice);

        switch (choice) {
            case 1: diagonalDistinct(n, a); break;
            case 2: diagonalSum(n, a); break;
            case 3: diagonalTraversal(n, a); break;
            case 4: printf("Exiting the program.\n"); break;
        }
    } while (choice != 4);

    return 0;
}

/* The main diagonal is a[0][0], a[1][1] and so on, so one index does
for both places. j starts from i + 1 so that each pair is checked
once, and so that an element is never compared with itself. */
void diagonalDistinct(int n, int a[n][n])
{
    int i, j, distinct = 1;

    printf("Main diagonal elements: ");
    for (i = 0; i < n; i++)
        printf("%d ", a[i][i]);
    printf("\n");

    for (i = 0; i < n; i++)
        for (j = i + 1; j < n; j++)
            if (a[i][i] == a[j][j])
                distinct = 0;

    if (distinct)
        printf("All the main diagonal elements are distinct\n");
    else
        printf("The main diagonal elements are not all distinct\n");
}
```

```

/* Only the diagonal is touched here, so one loop is enough. */
void diagonalSum(int n, int a[n][n])
{
    int i, sum = 0;

    for (i = 0; i < n; i++)
        sum += a[i][i];

    printf("Sum of main diagonal elements: %d\n", sum);
}

/* Every element on one diagonal has the same value of i + j, so d runs
   from 0 to 2n - 2 and picks out one diagonal at a time, and the column
   is just d - i. low and high keep the row inside 0 and n - 1.
   An even d counts the row down, which walks up and to the right, and
   an odd d counts it up, which walks down and to the left. I had these
   two the wrong way round at first and the output began 1 4 2 instead
   of 1 2 4, which is how I found it. */
void diagonalTraversal(int n, int a[n][n])
{
    int d, i, low, high;

    printf("Diagonal traversal: ");
    for (d = 0; d <= 2 * n - 2; d++) {
        low = (d - n + 1 > 0) ? d - n + 1 : 0;
        high = (d < n - 1) ? d : n - 1;

        if (d % 2 == 0)
            for (i = high; i >= low; i--)
                printf("%d ", a[i][d - i]);
        else
            for (i = low; i <= high; i++)
                printf("%d ", a[i][d - i]);
    }
    printf("\n");
}

```

5. Compilation and Output

I compiled it with `gcc -Wall -Wextra diagonal.c -o diagonal` so that even the small warnings would show up, and it compiled clean in the first attempt itself.

The Example Matrix from the Question

```
adilshakil — -zsh — 123x33

adilshakil@ADILs-MacBook-Air MyAssignment % gcc -Wall -Wextra diagonal.c -o diagonal
adilshakil@ADILs-MacBook-Air MyAssignment % ./diagonal
Enter the order of the square matrix: 3
Enter the matrix elements:
1 2 3
4 5 6
7 8 9

MENU
1. Check whether all the main diagonal elements are distinct
2. Find the sum of the main diagonal elements
3. Display the elements using a diagonal traversal
4. Exit
Enter your choice: 1
Main diagonal elements: 1 5 9
All the main diagonal elements are distinct

MENU
1. Check whether all the main diagonal elements are distinct
2. Find the sum of the main diagonal elements
3. Display the elements using a diagonal traversal
4. Exit
Enter your choice: 2
Sum of main diagonal elements: 15
█
```

The Traversal, and a Matrix with a Repeated Diagonal Element

```
adilshakil — -zsh — 123x33

Enter your choice: 3
Diagonal traversal: 1 2 4 7 5 3 6 8 9

MENU
1. Check whether all the main diagonal elements are distinct
2. Find the sum of the main diagonal elements
3. Display the elements using a diagonal traversal
4. Exit
Enter your choice: 4
Exiting the program.
adilshakil@ADILs-MacBook-Air MyAssignment % ./diagonal
Enter the order of the square matrix: 3
Enter the matrix elements:
1 2 3
4 1 6
7 8 9

MENU
1. Check whether all the main diagonal elements are distinct
2. Find the sum of the main diagonal elements
3. Display the elements using a diagonal traversal
4. Exit
Enter your choice: 1
Main diagonal elements: 1 1 9
The main diagonal elements are not all distinct

MENU
1. Check whether all the main diagonal elements are distinct
2. Find the sum of the main diagonal elements
3. Display the elements using a diagonal traversal
4. Exit
Enter your choice: 2
Sum of main diagonal elements: 11
█
```

A 4×4 Matrix, and Exit

```
adilshakil — -zsh — 123x33

adilshakil@ADILs-MacBook-Air MyAssignment % ./diagonal
Enter the order of the square matrix: 4
Enter the matrix elements:
1 2 3 4
5 6 7 8
9 10 11 12
13 14 15 16

MENU
1. Check whether all the main diagonal elements are distinct
2. Find the sum of the main diagonal elements
3. Display the elements using a diagonal traversal
4. Exit
Enter your choice: 2
Sum of main diagonal elements: 34

MENU
1. Check whether all the main diagonal elements are distinct
2. Find the sum of the main diagonal elements
3. Display the elements using a diagonal traversal
4. Exit
Enter your choice: 3
Diagonal traversal: 1 2 5 9 6 3 4 7 10 13 14 11 8 12 15 16

MENU
1. Check whether all the main diagonal elements are distinct
2. Find the sum of the main diagonal elements
3. Display the elements using a diagonal traversal
4. Exit
Enter your choice: 4
Exiting the program.
adilshakil@ADILs-MacBook-Air MyAssignment %
```

All eight test cases came out right, and the 3×3 run matches the example in the question exactly: 1 5 9 on the diagonal, 15 as the sum and 1 2 4 7 5 3 6 8 9 as the traversal.

The traversal took me the longest. What made it click is that every element sitting on one diagonal has the same value of $i + j$, so a single counter d from 0 to $2n - 2$ picks the diagonals off one after another and the column falls out as $d - i$. After that only two things were left. The row must not wander outside 0 and $n - 1$, which is what the two limits are for, and the direction has to flip on every diagonal. I had the flip the wrong way round at first, so the output started 1 4 2 instead of 1 2 4, and swapping the two loops fixed it. I also ran a 4×4 matrix, because on a 3×3 one a wrong rule can still look right.